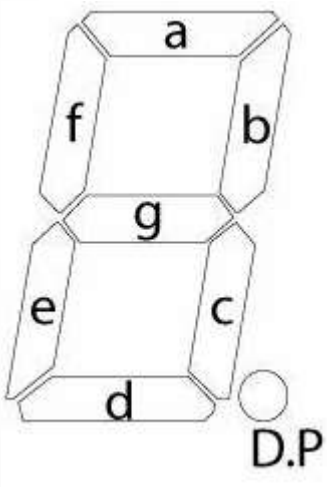
Circuitos Digitais (Práticas)

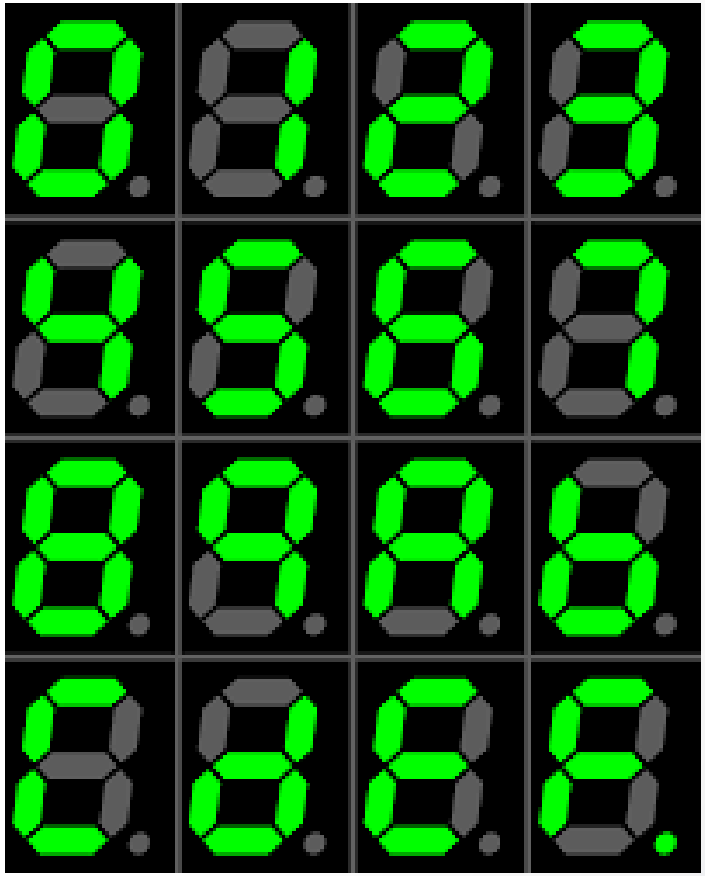
Prática 3

Prof. Gortan

Introdução

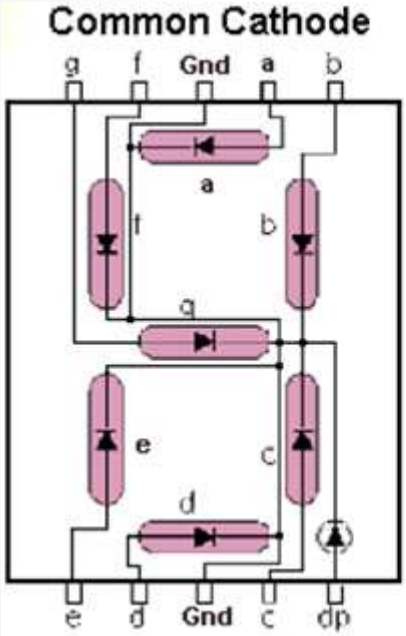
- *Display* de 7 Segmentos

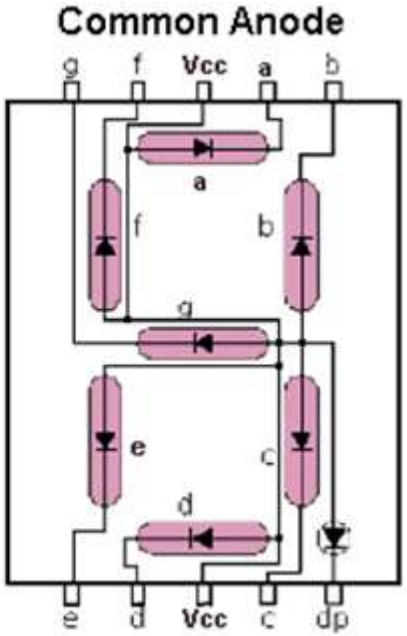


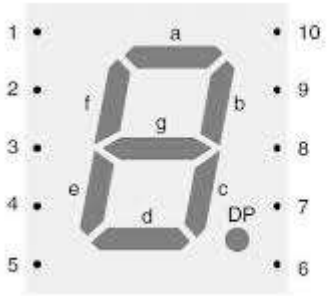


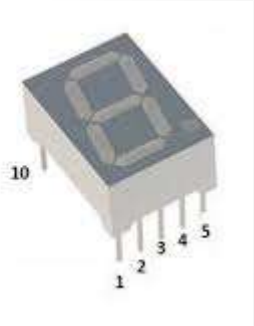
Introdução

- *Display* Catodo Comum e Anodo Comum



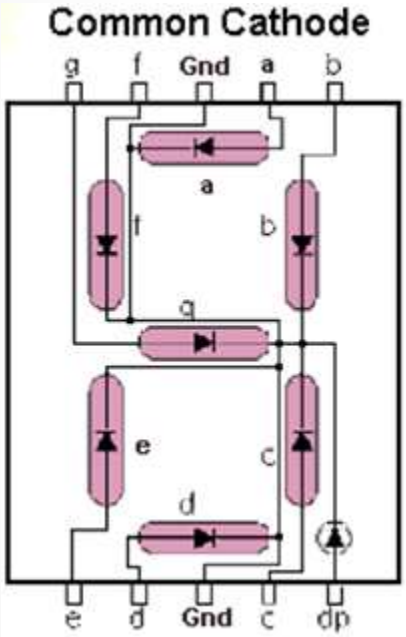


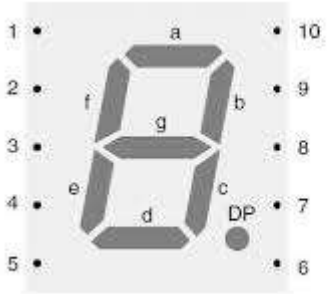


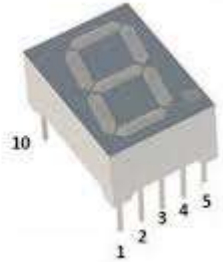


Introdução

- Nesta aula usaremos Catodo Comum





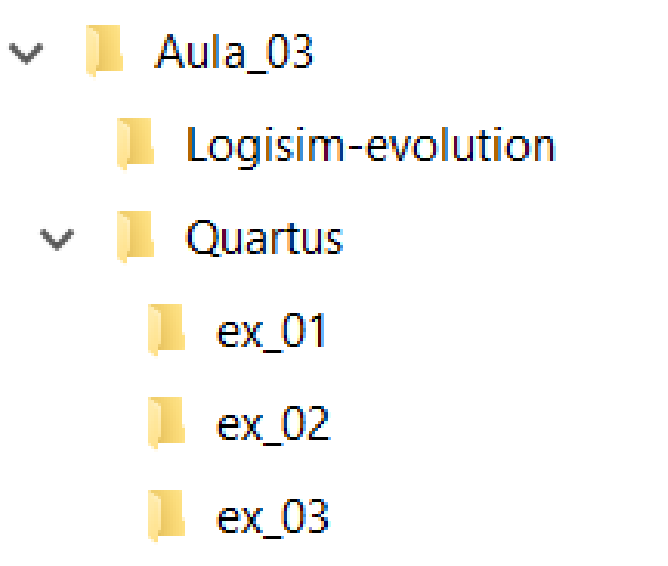


Atenção!.: ***Lembrar de colocar os resistores de 100Ω a 330Ω para limitar a corrente de cada LED do display se você está montando seus testes no protoboard.***

A placa adicional de displays já contém esses resistores.

Organização dos arquivos gerados nesta aula

- Sugere-se que você crie uma pasta de trabalho chamada *Aula_03*, com dentro duas subpastas: *Logisim-evolution* e *Quartus*
- A pasta Quartus, por sua vez terá 3 subpastas: *ex_01*, *ex_02* e *ex_03*



Exercício 1

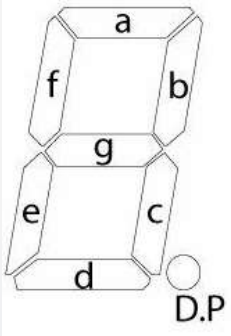
Circuito conversor de
binário para display 7 segmentos

Entrada binária: 4 bits – 0000 até 1111
Saída: 7 segmentos *a* até *g*

Exercício 1 – 1ª Parte

Desenvolvimento da lógica do decodificador de 7 segmentos testes no Logisim-evolution

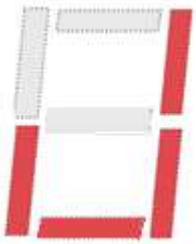
Antes de mais nada:

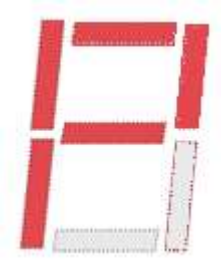


K	L	M	N	a	b	c	d	e	f	g
0	0	0	0	?	?	?	?	?	?	?
0	0	0	1	?	?	?	?	?	?	?
0	0	1	0	?	?	?	?	?	?	?
0	0	1	1	?	?	?	?	?	?	?
0	1	0	0	?	?	?	?	?	?	?
0	1	0	1	?	?	?	?	?	?	?
0	1	1	0	?	?	?	?	?	?	?
0	1	1	1	?	?	?	?	?	?	?
1	0	0	0	?	?	?	?	?	?	?
1	0	0	1	?	?	?	?	?	?	?
1	**0**	**1**	**0**	**?**	**?**	**?**	**?**	**?**	**?**	**?**
1	**0**	**1**	**1**	**?**	**?**	**?**	**?**	**?**	**?**	**?**
1	**1**	**0**	**0**	**?**	**?**	**?**	**?**	**?**	**?**	**?**
1	**1**	**0**	**1**	**?**	**?**	**?**	**?**	**?**	**?**	**?**
1	1	1	0	?	?	?	?	?	?	?
1	1	1	1	?	?	?	?	?	?	?

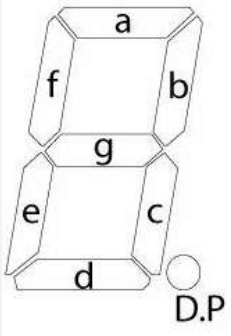
Preencha a tabela da verdade ao lado com os valores válidos para sua equipe (veja exemplo como fazer no próximo slide):

- Hexadecimais de 0 a D deverão mostrar os valores 0 a D
- Hexadecimal E: deverá mostrar o último dígito do seu RA.
- Hexadecimal F:
  RA ímpares: mostra letra ***J***
  RA pares: mostra letra ***P***





Exemplo: segmento a



K	L	M	N	a	b	c	d	e	f	g
0	0	0	0	**1**	?	?	?	?	?	?
0	0	0	1	**0**	?	?	?	?	?	?
0	0	1	0	**1**	?	?	?	?	?	?
0	0	1	1	**1**	?	?	?	?	?	?
0	1	0	0	**0**	?	?	?	?	?	?
0	1	0	1	**1**	?	?	?	?	?	?
0	1	1	0	**1**	?	?	?	?	?	?
0	1	1	1	**1**	?	?	?	?	?	?
1	0	0	0	**1**	?	?	?	?	?	?
1	0	0	1	**1**	?	?	?	?	?	?
1	**0**	**1**	**0**	**1**	**?**	**?**	**?**	**?**	**?**	**?**
1	**0**	**1**	**1**	**0**	**?**	**?**	**?**	**?**	**?**	**?**
1	**1**	**0**	**0**	**1**	**?**	**?**	**?**	**?**	**?**	**?**
1	**1**	**0**	**1**	**0**	**?**	**?**	**?**	**?**	**?**	**?**
1	1	1	0	**1**	?	?	?	?	?	?
1	1	1	1	**1**	?	?	?	?	?	?

A coluna do segmento ‘a’ ao lado foi preenchida como exemplo, para implementar os números hexa de 0 a F como no slide 2.

O resultado, lendo os bits de baixo para cima e agrupando de 4 em 4 fica:

- 1101–0111–1110–1101
- Em hexa: d – 7 – e – d

Logisim-evolution

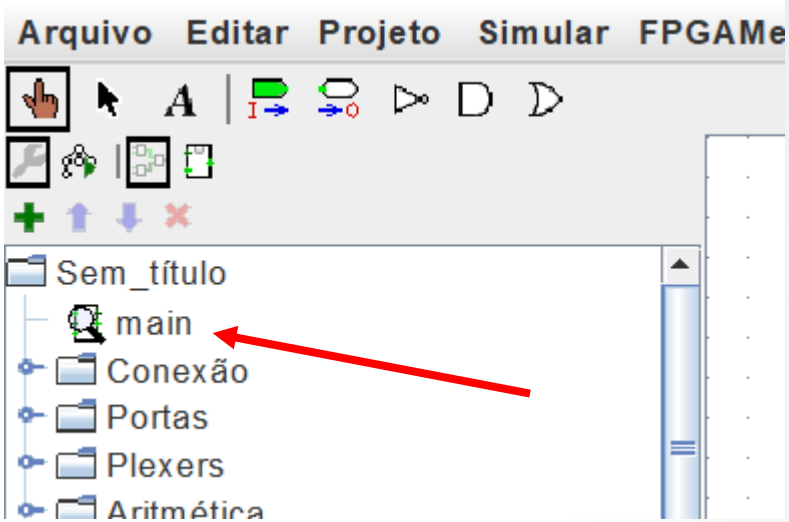
No Logisim-evolution, monte um decodificador de decimal para 7 segmentos usando 7 muxes de 16 entradas e uma saída cada. Faça isso em um subcircuito que fará a função de decodificador.

Uma maneira de implementar uma tabela da verdade de 4 entradas e uma saída é usando um MUX de 16 entradas e uma saída, controlado por 4 bits, que selecionam qual entrada é conectada à saída.

Os próximos slides dão uma dica de como fazer isso.

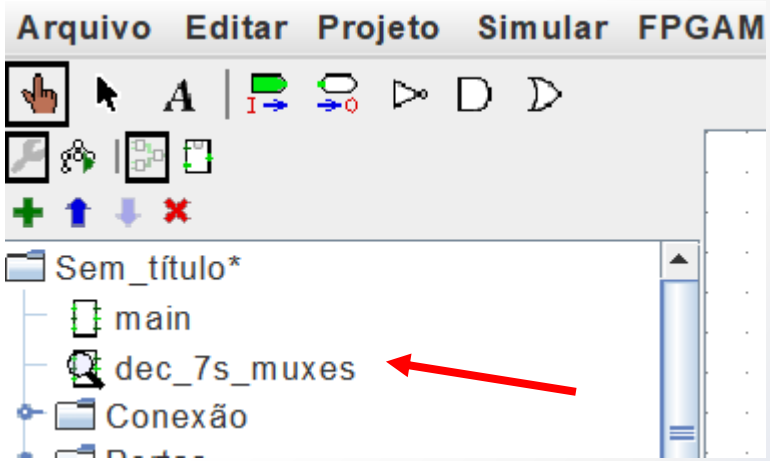
Logisim-evolution

Quando você abre o Logisim-evolution, aparece um circuito em branco, chamado "*main*":



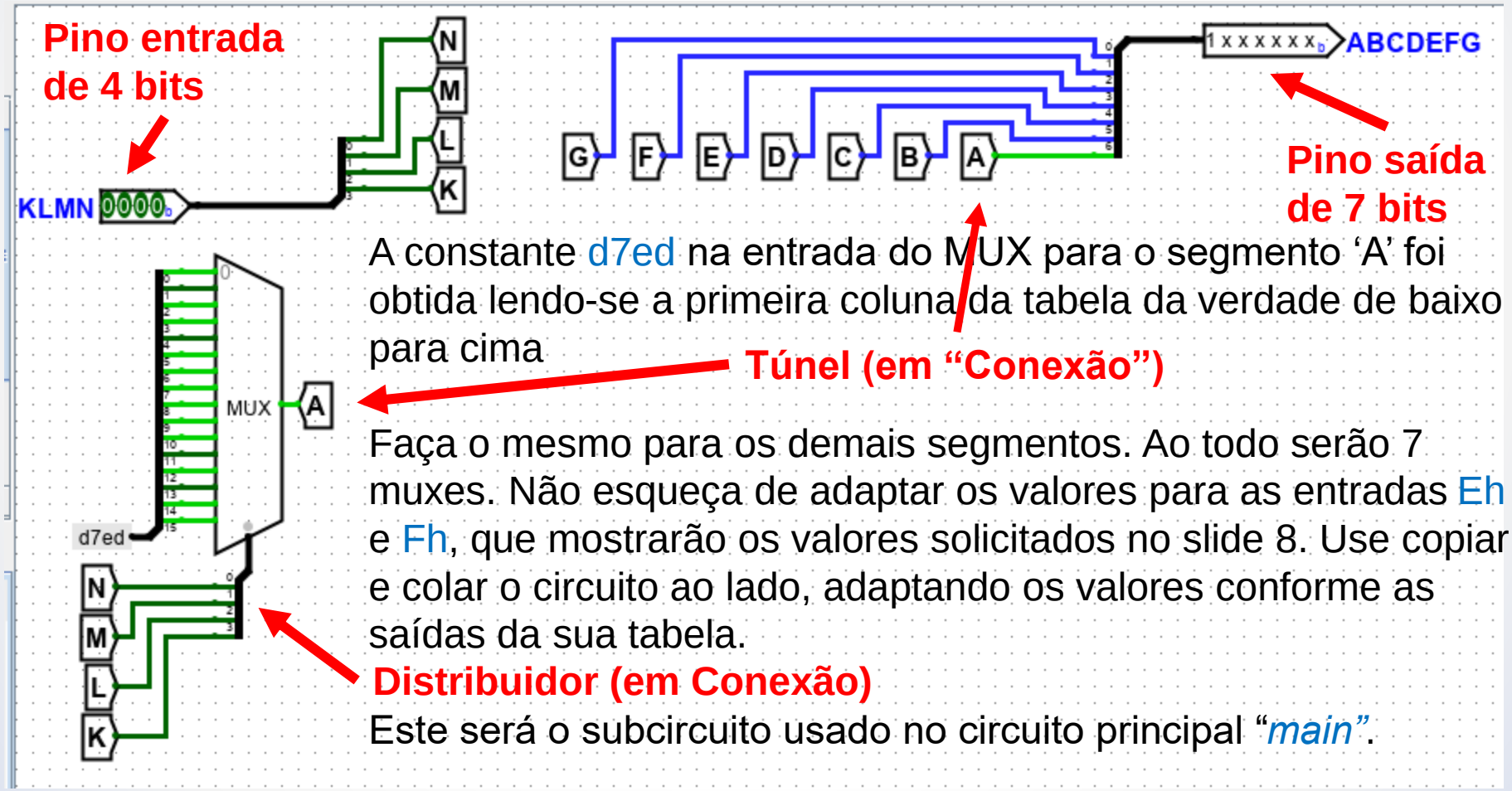
Não use ainda o "*main*".

Usando o menu "***Projeto/acrescentar circuito...***" crie um subcircuito "*dec_7s_muxes*".



Logisim-evolution

Subcircuito "*dec_7s_muxes*" (parcial – só segmento 'A').
Complete o subcircuito para os segmentos B até G:



A constante d7ed na entrada do MUX para o segmento 'A' foi obtida lendo-se a primeira coluna da tabela da verdade de baixo para cima

Túnel (em "Conexão")

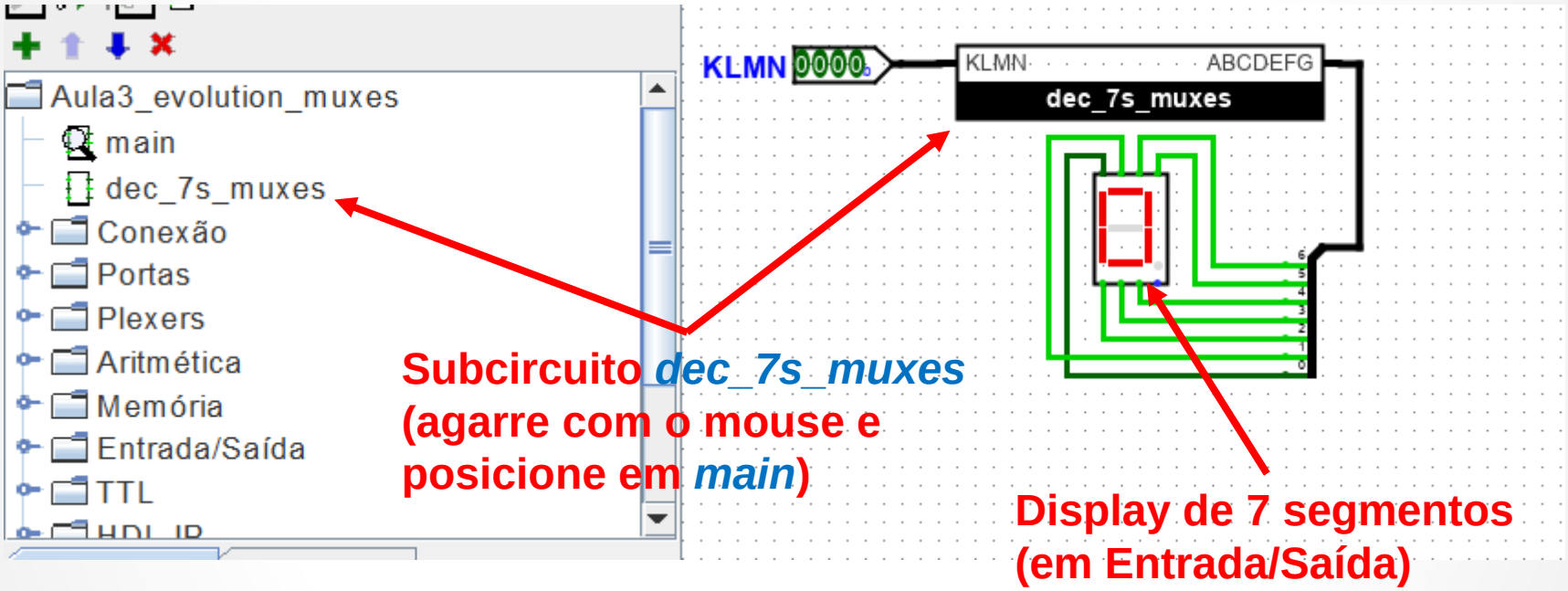
Faça o mesmo para os demais segmentos. Ao todo serão 7 muxes. Não esqueça de adaptar os valores para as entradas Eh e Fh, que mostrarão os valores solicitados no slide 8. Use copiar e colar o circuito ao lado, adaptando os valores conforme as saídas da sua tabela.

Distribuidor (em Conexão)

Este será o subcircuito usado no circuito principal "*main*".

Logisim-evolution

No circuito main use o subcircuito *dec_7s_muxes*:



Teste o circuito para certificar-se que produz os resultados solicitados para sua equipe.

Logisim-evolution

Ainda no Logisim-evolution, monte um bloco HDL que faça a mesma função do decodificador e teste ambos em paralelo

Baseie-se, para isso, nos slides da teoria, onde foi mostrado como implementar uma tabela da verdade em VHDL usando *with/select*.

O próximo slide mostra (parcialmente) como usar esses comandos com um vetor de saída de 7 posições e um vetor de entrada de 4 posições

Logisim-evolution

Complete o código VHDL para o decodificador

```
entity dec_7s is
  port(
  ------------------------------------------------------
  --Entradas (4 bits)
    cnt_i       : in  std_logic_vector(3 downto 0);   --
  ------------------------------------------------------
  --Saídas (7 bits)
    c_7s        : out std_logic_Vector(6 downto 0)    --
    );
end dec_7s;
```

```
architecture decode of dec_7s is


begin
```

Saídas a até g Entradas klmn

```
----------------abcdefg-------klmn------------------

with cnt_i select
    c_7s <=   "1111110" when "0000",   -- 0 - 7E h
              "0110000" when "0001",   -- 1 - 30 h
              "1101101" when "0010",   -- 2 - 6D h
              "1111001" when "0011",   -- 3 - 79 h
```

Exercício 1 – 2ª Parte

Desenvolvimento do projeto no Quartus
Simulação no Quartus

Quartus

- Exporte o VHDL do Logisim-evolution e monte um projeto no Quartus (pasta ex_01).
- Compile e simule o projeto no Quartus
- Verifique a implementação RTL (Register Transfer Logic) do quartus e faça um print da mesma.
- Simule o projeto no Quartus como já aprendido na aula anterior. Guarde um print da simulação.

Quartus

- Exporte o VHDL do Logisim-evolution e monte um projeto no Quartus (pasta *ex_01*).
- Chame o arquivo vhdl exportado de *dec_7s.vhd*.
- Crie um arquivo de configurações (assignments) base como nas aulas anteriores, que contenha a atribuição de pinos para esse projeto.
- Chame esse arquivo de *dec_7s_base.qsf*

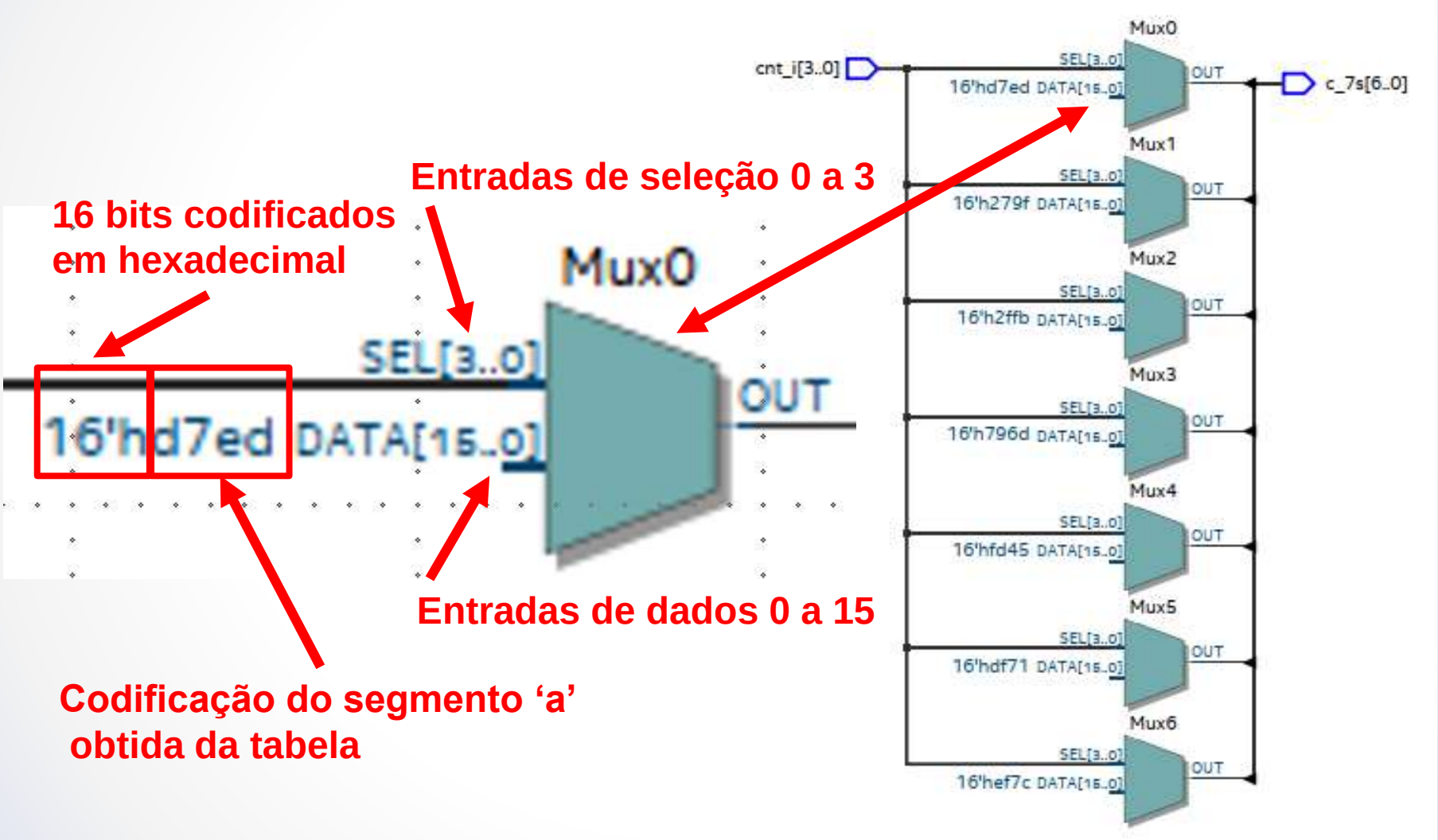
Quartus

- Você pode obter a implementação RTL do Quartus clicando em

  Tools/NetlistViewers/RTL Viewer

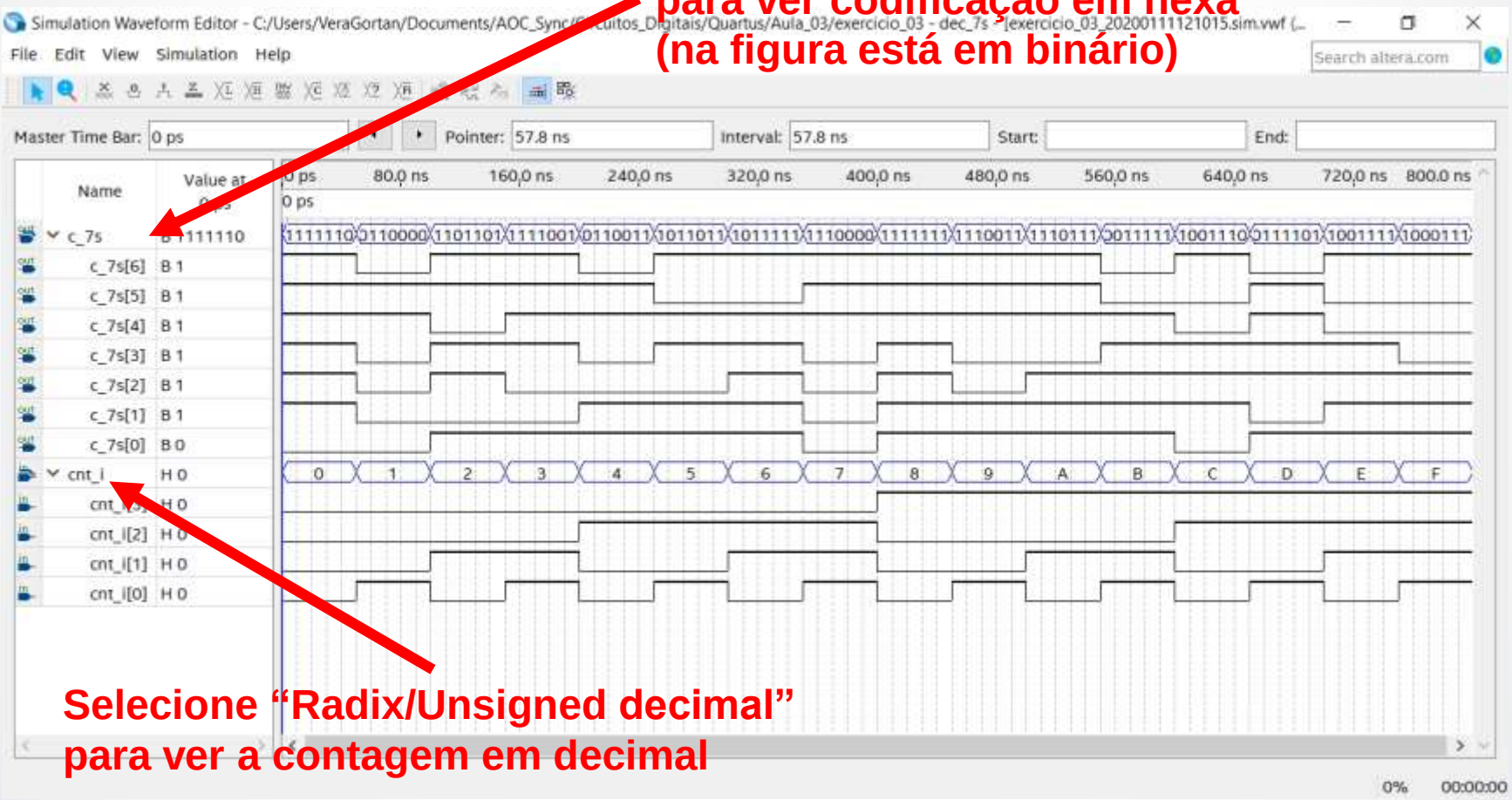
- O resultado deve ser semelhante à figura do próximo slide, embora cada equipe deva ter dados diferentes

Quartus – RTL


Entradas de seleção 0 a 3
16 bits codificados em hexadecimal
Mux0
SEL[3..0]
16'hd7ed
DATA[15..0]
OUT
Entradas de dados 0 a 15
Codificação do segmento 'a' obtida da tabela
cnt_i[3..0]
c_7s[6..0]
Mux0
16'hd7ed
Mux1
16'h279f
Mux2
16'h2ffb
Mux3
16'h796d
Mux4
16'hfd45
Mux5
16'hdf71
Mux6
16'hef7c

Quartus – Simulação

Selecione "Radix/hexadecimal" para ver codificação em hexa (na figura está em binário)



Selecione "Radix/Unsigned decimal" para ver a contagem em decimal

Exercício 1 – 3ª Parte

Montagem do Circuito
no Kit de Desenvolvimento
Testes de Verificação

Kit EPM240

- Atribua entradas e saídas ao projeto do Quartus (se ainda não fez), usando um arquivo de base e importando as configurações (como nas aulas anteriores).

- Conecte a placa adicional de diplay ao kit e teste todas as 16 combinações de entrada. Lembre-se:
  - Tecla pressionada = GND (0)
  - Tecla não pressionada = 3.3 V (1) (se configurado pull-up na entrada)

- Não se esqueça de conectar o GND do kit ao GND da placa adicional (o cabo flat não tem essa conexão).

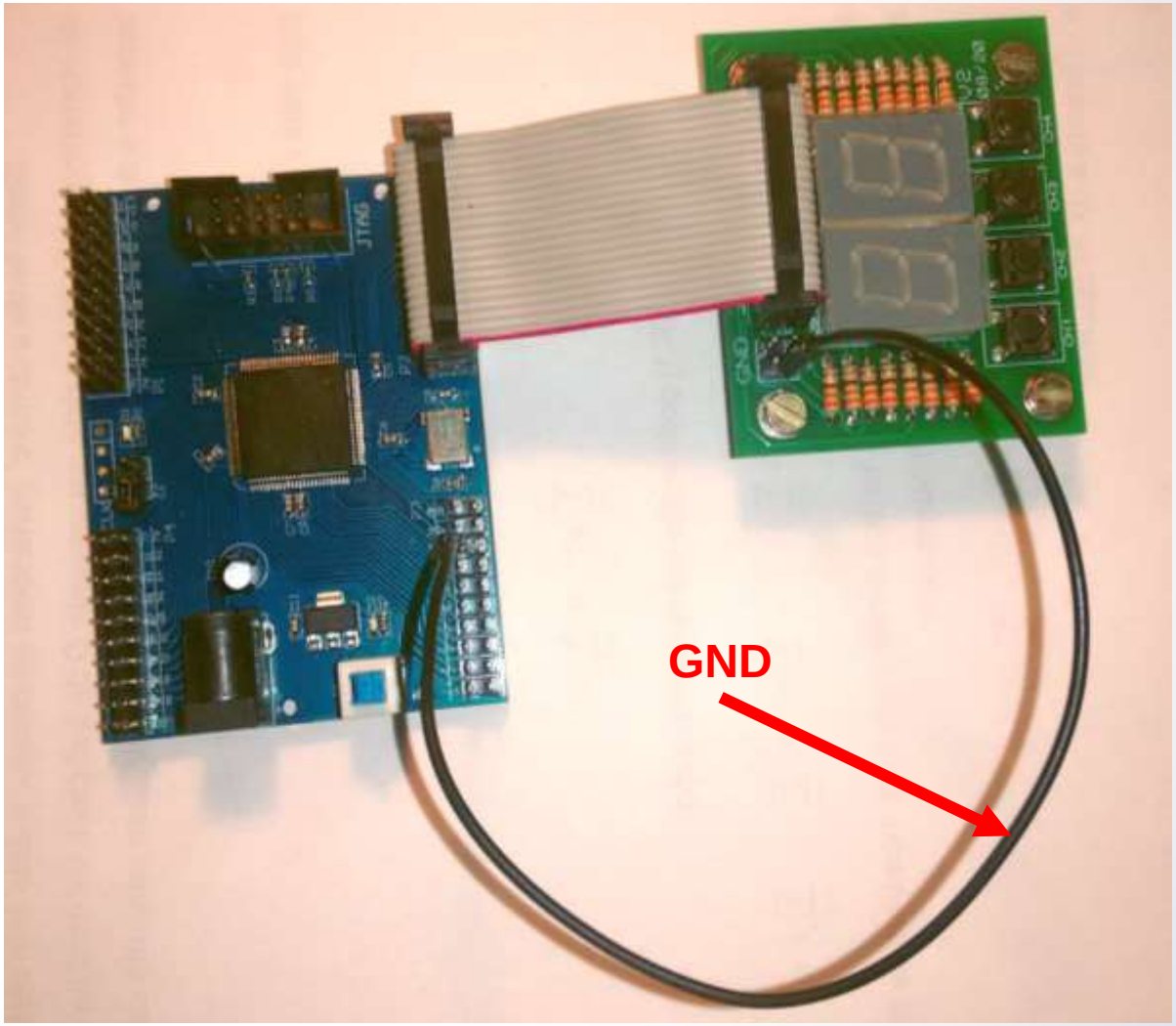
Kit EPM240

Obs.:
As teclas comutam GND ou aberto.

Então é preciso conectar resistores de pull-up nas entradas de teclas para garantir o nível ‘1’ com a tecla não pressionada.

Porém não é necessário usar resistores externos.

Basta configurar as entradas do EPM240 para conectar os resistores de pull-up internos do CI.



Exercício 2

Complementação do circuito conversor de binário para display 7 segmentos para avançar uma posição a cada pulso

Entrada de pulso: 1 bit
Saída: 7 segmentos *a* até *g*

Exercício 2 – 1ª Parte

Complementação da lógica do decodificador de 7 segmentos no Logisim-evolution

Logisim-evolution

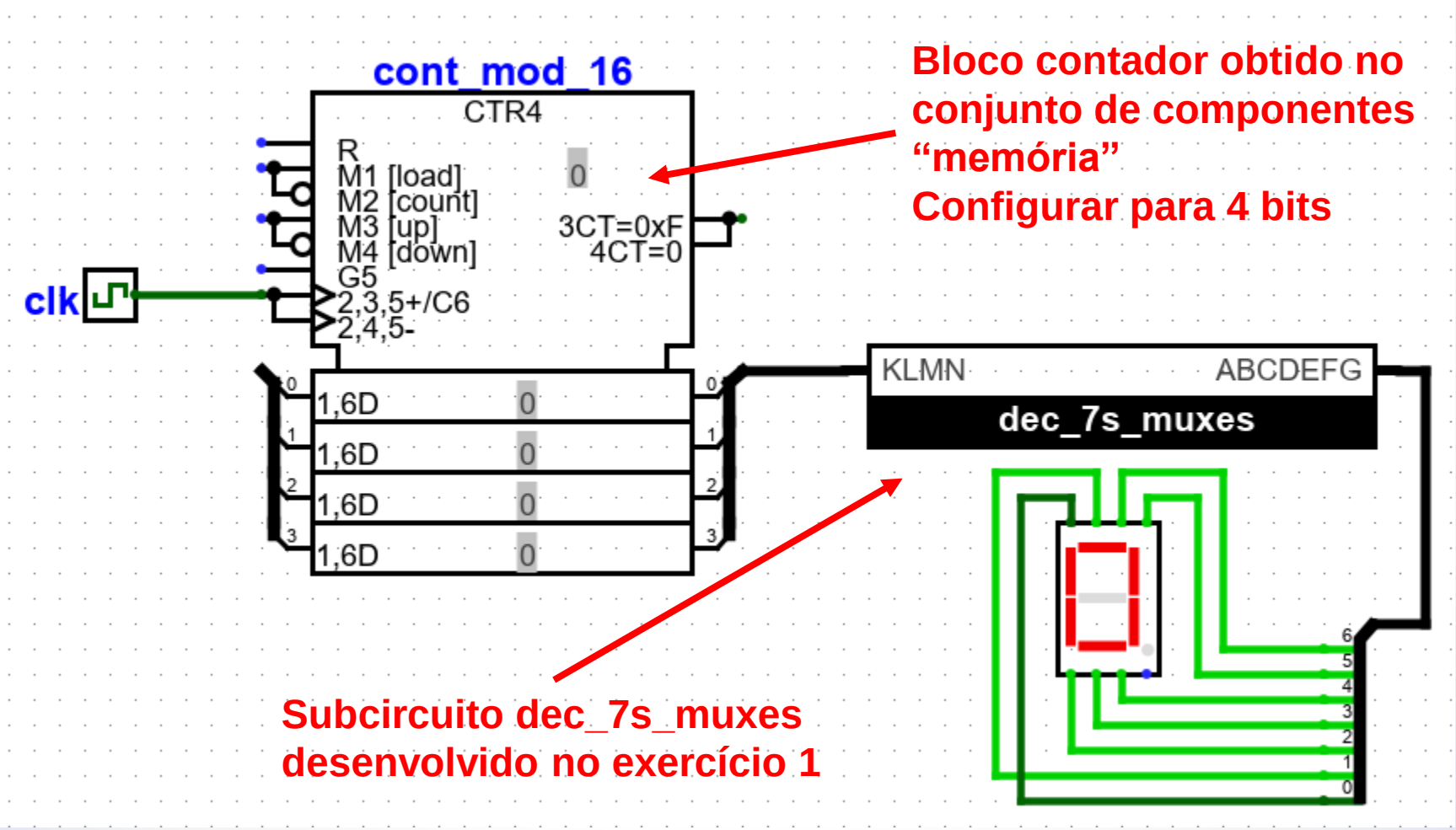
- Faça uma cópia do Exercício 1 do Logisim-evolution e renomeie-a para Exercício 2.

- Acrescente ao decodificador um contador de 4 bits que fará o decodificador avançar uma posição a cada pulso de entrada.

Os próximos slides dão uma dica de como fazer isso.

Logisim-evolution

Conversor 7 segmentos com contador:



Logisim-evolution

Observe que, no slide anterior, não desenvolvemos o circuito digital do contador.

Os contadores serão estudados em uma próxima aula, então por ora utilizamos um bloco pronto.

Em VHDL faremos a mesma coisa: utilizaremos um arquivo de contador pronto, disponibilizado pela altera na forma de IP ("*Intelectual Property*").

Prossiga então para a segunda parte do Exercício 2, onde será mostrado como usar o bloco contador IP da Altera em VHDL

Exercício 2 – 2ª Parte

Utilização de um circuito contador binário usando um bloco IP da Altera no Quartus

Quartus

O projeto do exercício 2 será criado na pasta *ex_02*.

Copie antes para essa pasta o arquivo *dec_7s.vhd* do exercício anterior e renomeie-o para *dec_7s_cnt.vhd*. Esse arquivo será depois alterado convenientemente.

Copie também o arquivo de configurações *dec_7s_base.qsf* do exercício anterior renomeando-o para *dec_7s_cnt_base.qsf*.

Altere o arquivo para ter somente duas entradas: uma chamada clk (*clock*) e outra chamada clr (*clear*) (serão as teclas CH1 e CH2).

Quartus

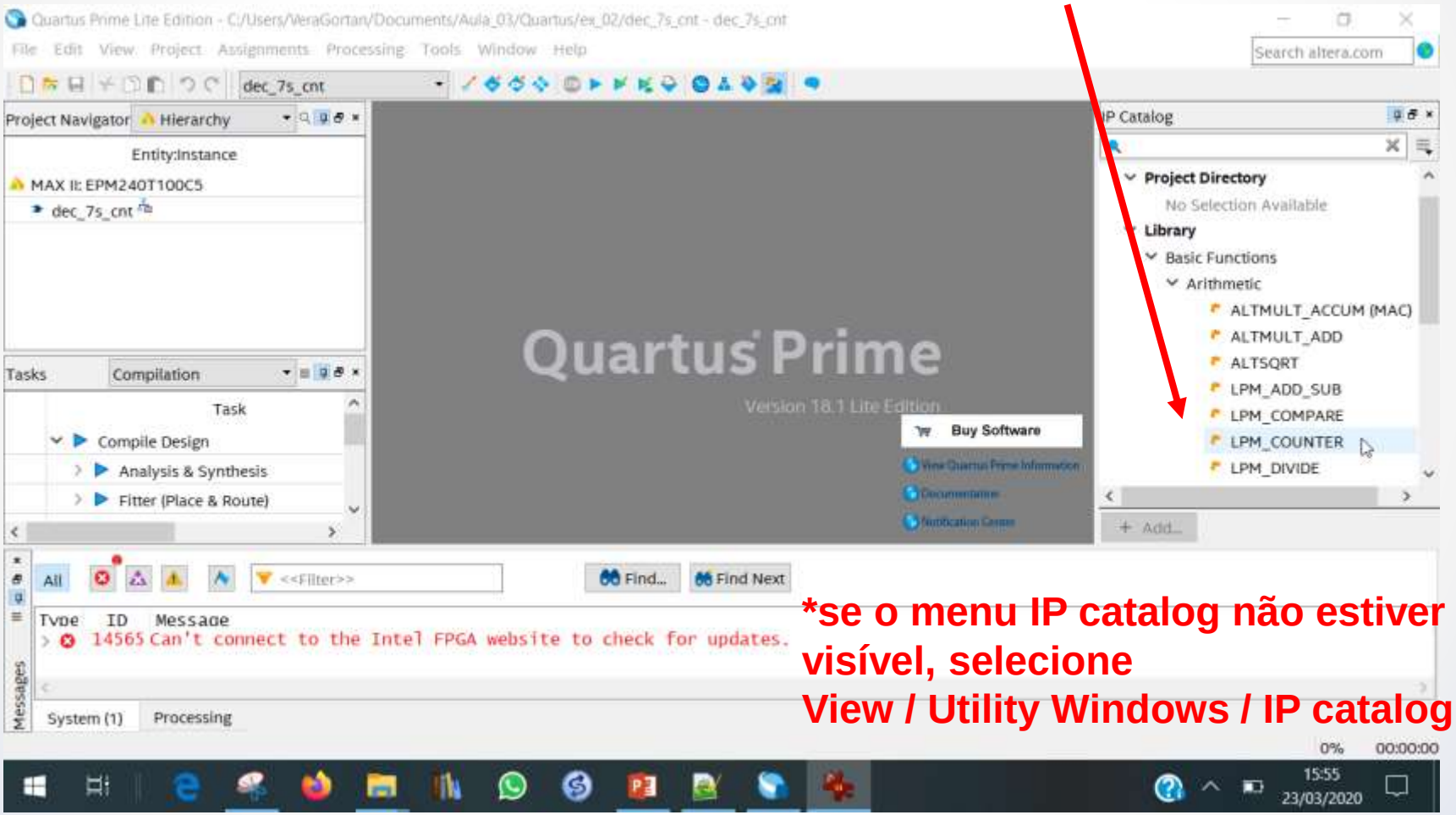
Você pode agora criar o projeto do exercício 02 no Quartus, incluindo nesse projeto o arquivo *dec_7s_cnt.vhd*.

Siga os passos indicados nos próximos slides para criar o arquivo vhdl do contador, a que chamaremos de *cnt_1s.vhd*.

Quando tiver concluído a criação desse arquivo, inclua-o também no projeto do exercício 02.

Quartus
Criação do arquivo contador usando bloco IP da Altera

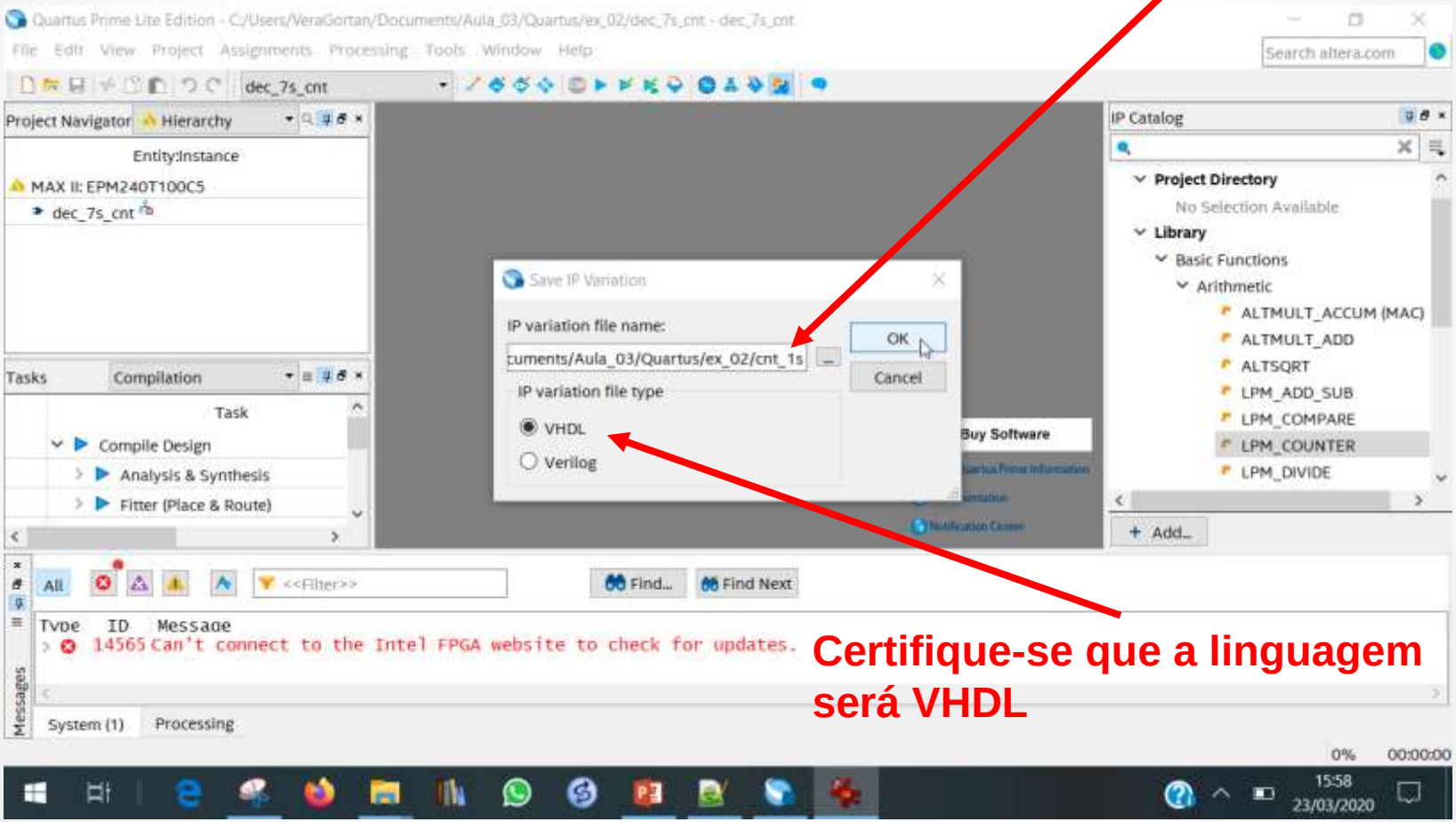
No menu IP Catalog selecione funções básicas/Aritmética/LPM_COUNTER:



***se o menu IP catalog não estiver visível, selecione View / Utility Windows / IP catalog**

Quartus
Criação do arquivo contador usando bloco IP da Altera

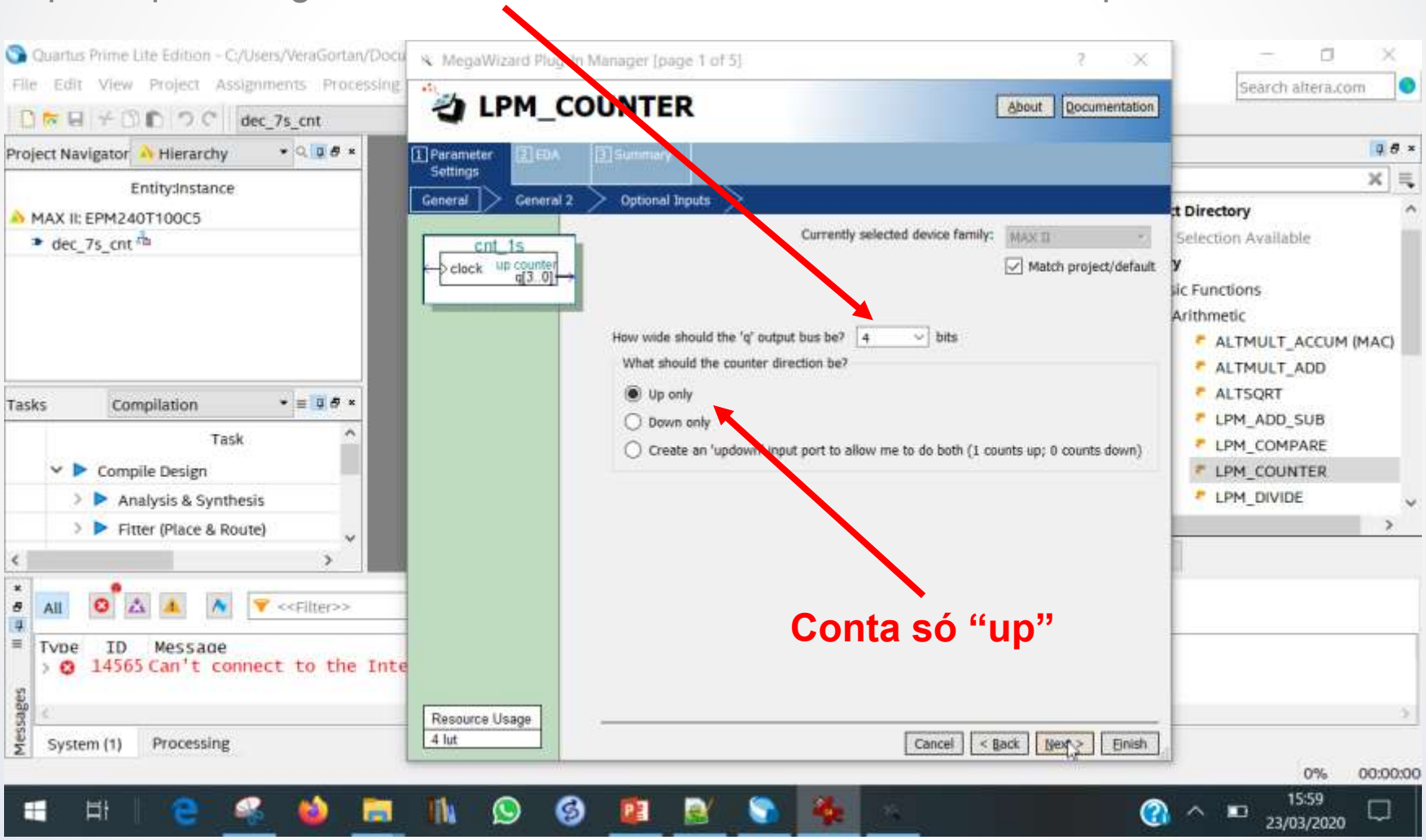
Forneça o nome desejado para o arquivo VHDL a ser criado (cnt_1s)



Certifique-se que a linguagem será VHDL

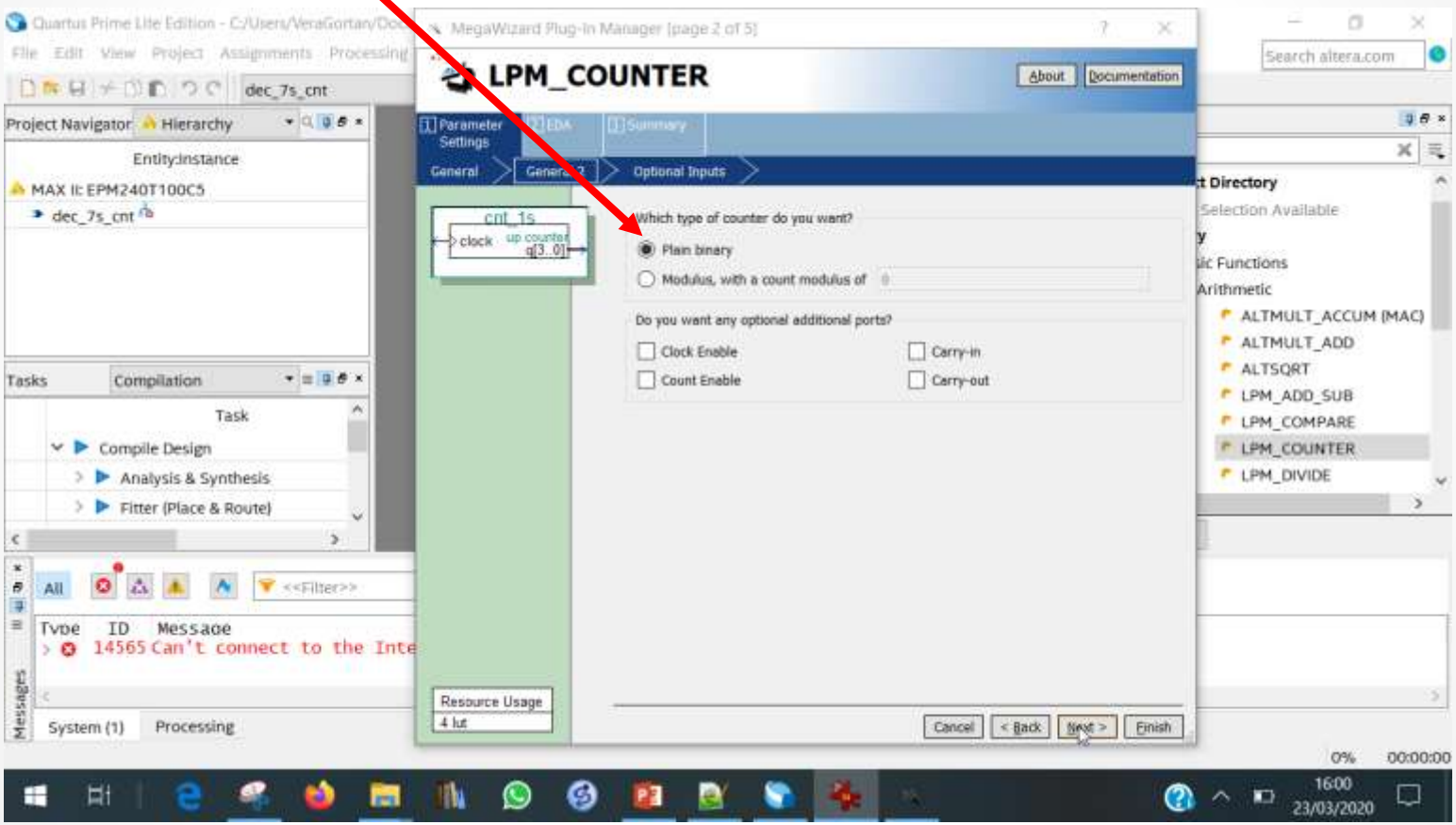
Quartus
Criação do arquivo contador usando bloco IP da Altera

Especifique a largura de 4 bits do contador e o modo “somente para cima”



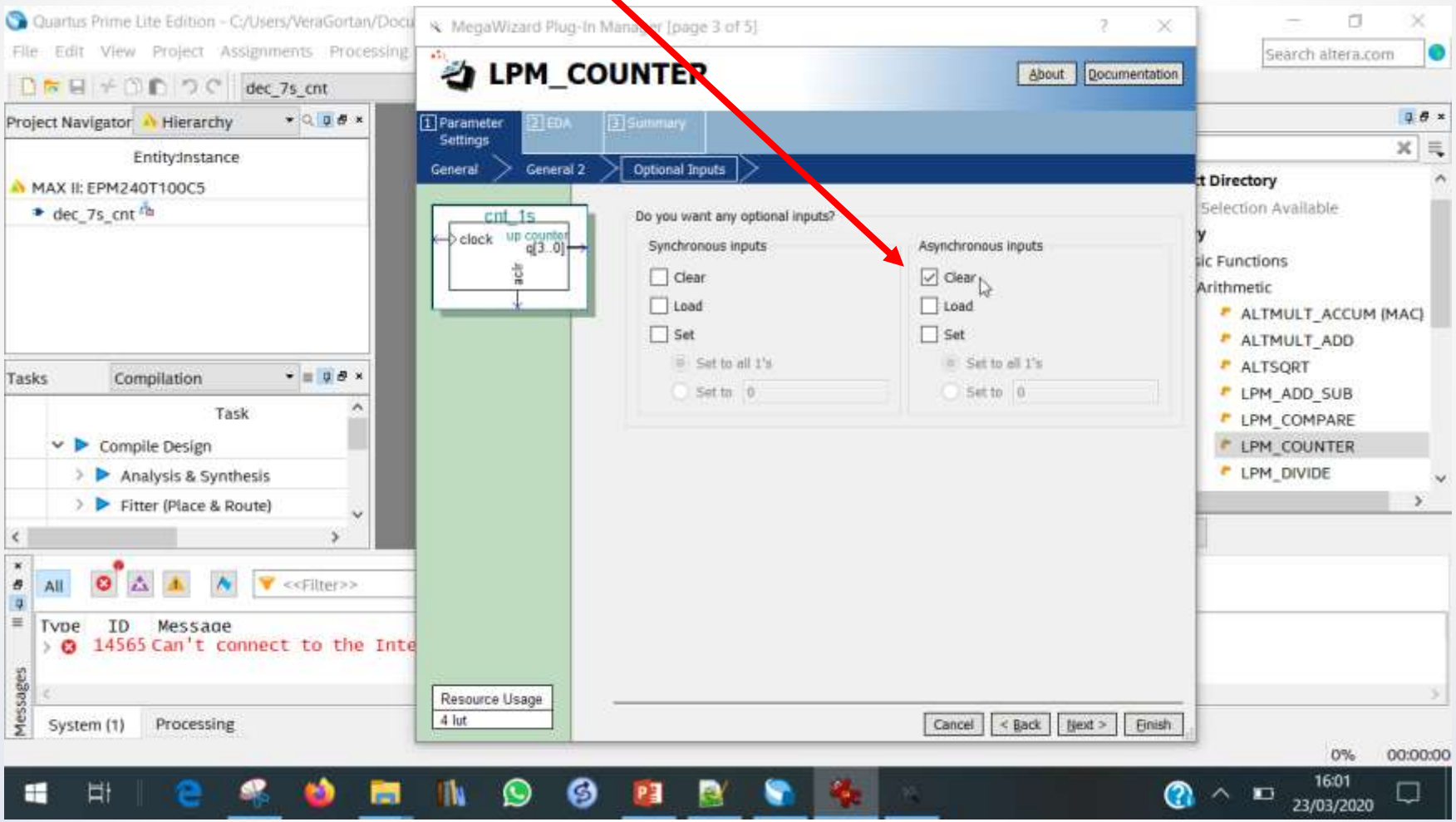
Quartus
Criação do arquivo contador usando bloco IP da Altera

Especifique “plain binary”, ou seja: módulo 16



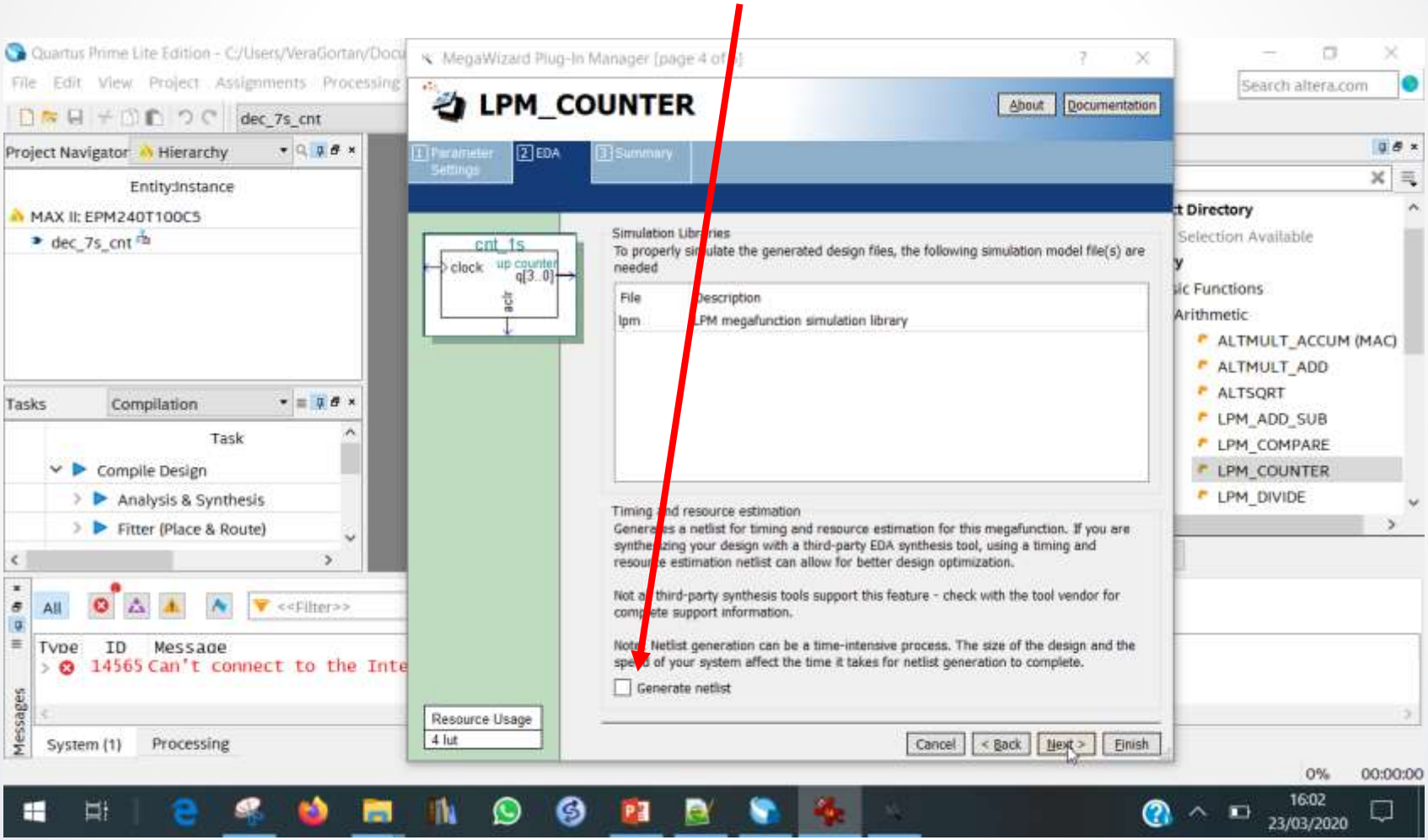
Quartus
Criação do arquivo contador usando bloco IP da Altera

Especifique uma entrada de “clear” assíncrona:



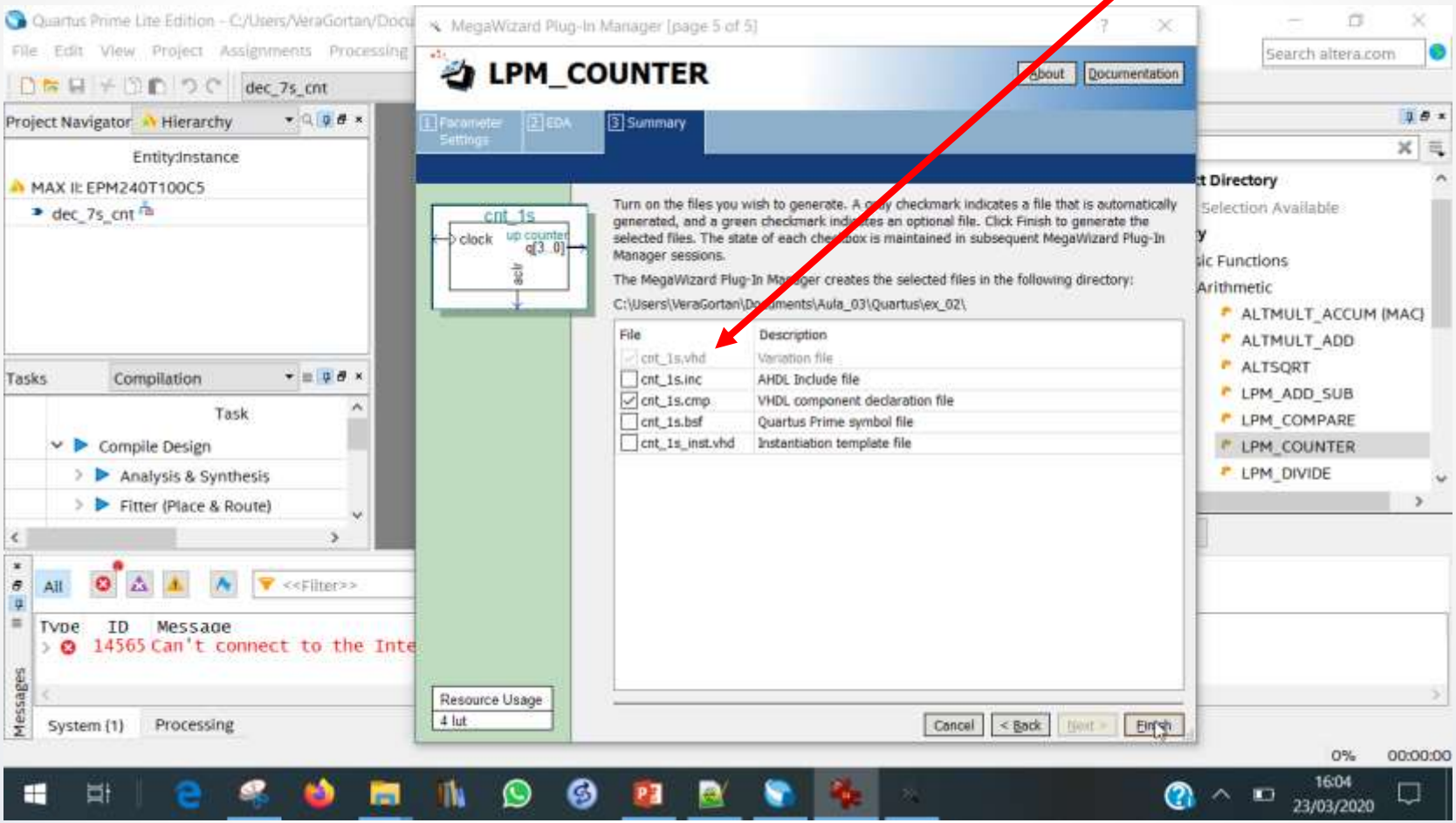
Quartus
Criação do arquivo contador usando bloco IP da Altera

Nada a fazer aqui. Deixe o item “generate netlist” em branco:



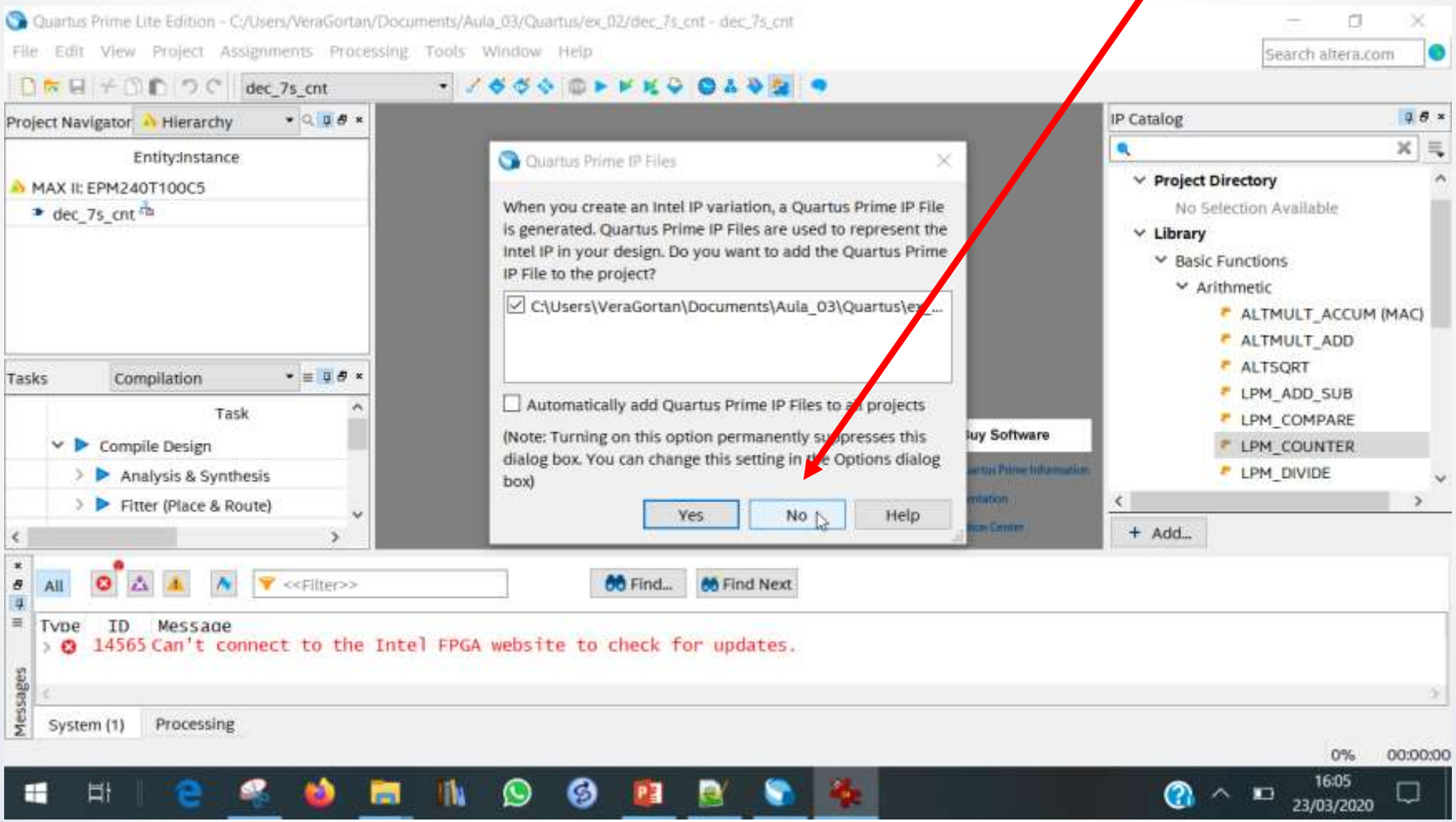
Quartus
Criação do arquivo contador usando bloco IP da Altera

Esta tela permite selecionar quais arquivos serão gerados cnt_1s.vhd já está



Quartus
Criação do arquivo contador usando bloco IP da Altera

Esta tela oferece para incluir o arquivo .qip no projeto – selecione não



Quartus
Criação do arquivo contador usando bloco IP da Altera

Pronto! O arquivo **cnt_1s.vhd** foi gerado!

Como você não aceitou a inclusão automática de arquivos na tela anterior, você deverá agora incluir esse arquivo no seu projeto:

Vá até o menu *Assignments / Settings / Files* e inclua o arquivo *cnt_1s.vhd*.

BTW: O *LPM* em LPM_COUNTER significa *Library of Parameterized Modules*, ou seja: *Biblioteca de Módulos Parametrizados ou Parametrizáveis*. Como você viu nas telas anteriores, o módulo LPM_COUNTER foi parametrizado por nós passo a passo.

Quartus
Conexão dos módulos dec_7s_cnt e cnt_1s

Siga os passos abaixo para modificar o arquivo dec_7s_cnt.vhd para que seu circuito fique conectado ao contador cnt_1s:

- Altere a "***entity***" do circuito, para ter somente duas entradas: clk e clr e ser chamada de *dec_7s_cnt*:

```
entity dec_7s_cnt is
  port(
  ------------------------------------------------------------------------------
  --Entradas (4 bits)
    clk          : in std_logic;
    clr          : in std_logic;
    --cnt_i       : in  std_logic_vector(3 downto 0);   -- vetor entrada binária
  ------------------------------------------------------------------------------
  --Saídas (7 bits)
    c_7s        : out std_logic_Vector(6 downto 0)   -- vetor com os 7 segmentos
    );
end dec_7s_cnt;
```

Quartus
Conexão dos módulos dec_7s_cnt e cnt_1s

- Crie um *signal* de 4 bits (cnt_i) que servirá para conectar a saída do módulo contador com a entrada do decodificador 7s.

- Declare o componente cnt_1s como abaixo.
  Obs.: o Quartus criou um arquivo cnt_1s.cmp que contém essa declaração. Você pode copiar e colar a declaração a partir desse arquivo, se quiser.

```
architecture decode of dec_7s_cnt is

signal cnt_i: std_logic_vector(3 downto 0);   -- vetor entrada binária

component cnt_1s is
    PORT
    (
        aclr     : IN STD_LOGIC ;
        clock    : IN STD_LOGIC ;
        q        : OUT STD_LOGIC_VECTOR (3 DOWNTO 0)
    );
end component;

begin
```

Quartus
Conexão dos módulos dec_7s_cnt e cnt_1s

Faça a conexão entre as entradas e saídas do módulo cnt_1s com o decodificador usando **port map** como abaixo:

- clock do cnt_1s é ligado na entrada clk do decodificador
- aclr do cnt_1s é ligado na entrada clr do decodificador
- q do cnt_1s é ligado no signal cnt_i, que agora é a entrada da decodificação

```
    );
end component;

begin
cnt: cnt_1s port map(clock => clk, aclr => clr, q => cnt_i);
----------------abcdefg-------klmn------------------

with cnt_i select
    c_7s <=   "1111110" when "0000",    -- 0 - 7E h
```

Quartus
Conexão dos módulos dec_7s_cnt e cnt_1s

Obs.: No slide anterior, no “port map” conectamos a entrada aclr do contador diretamente com a entrada clr, que está conectada a uma tecla.

aclr é ativo alto (não havia outra opção no IP da Altera), logo, com a tecla não pressionada o circuito estará sendo ressetado diretamente e você precisará manter a tecla pressionada para que ele possa contar pulsos.

Você pode alterar facilmente essa condição introduzindo um inversor entre clr e aclr no “port map”:

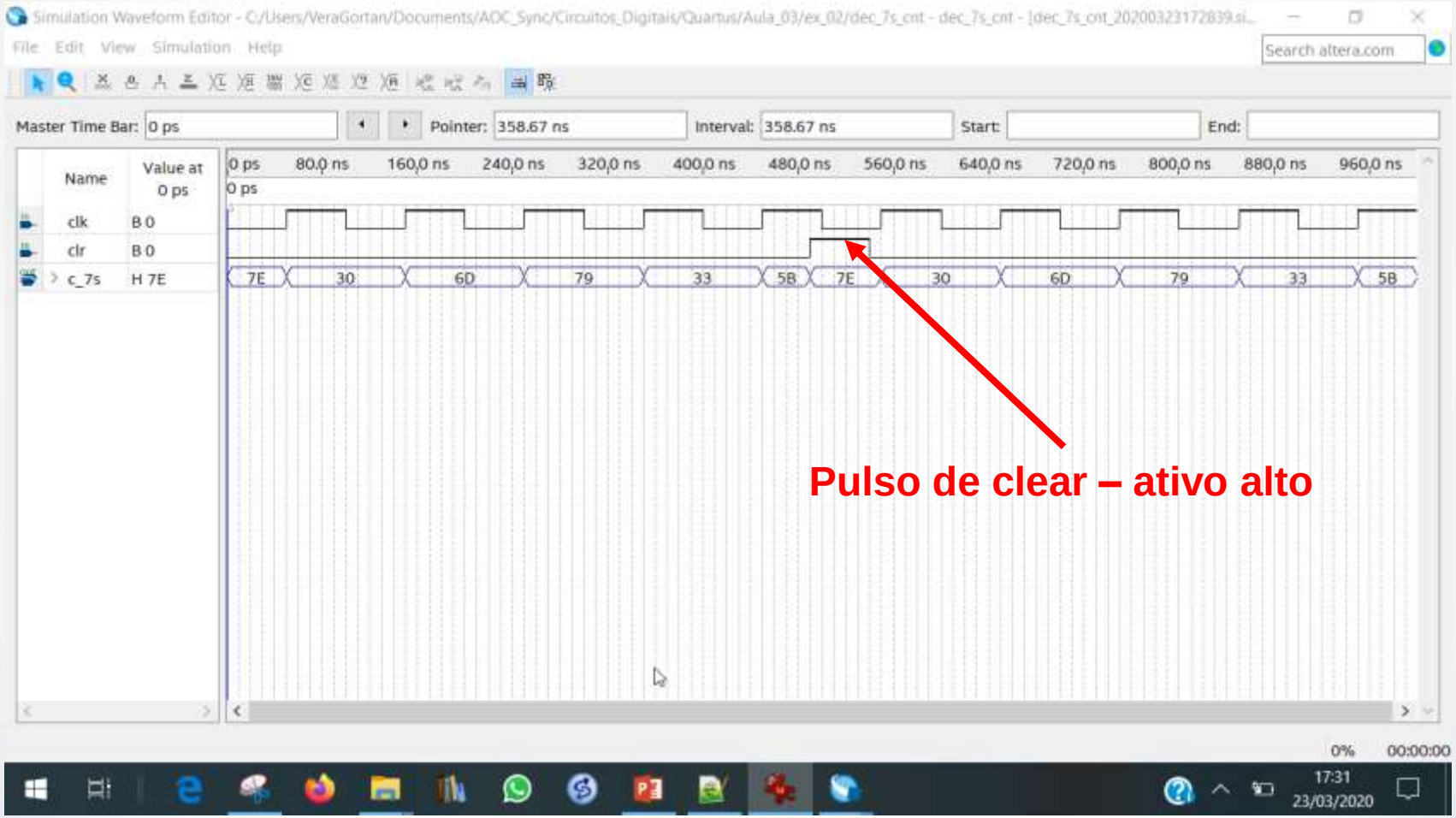
aclr => not clr

Com isso o circuito será ressetado somente ao pressionarmos a tecla.

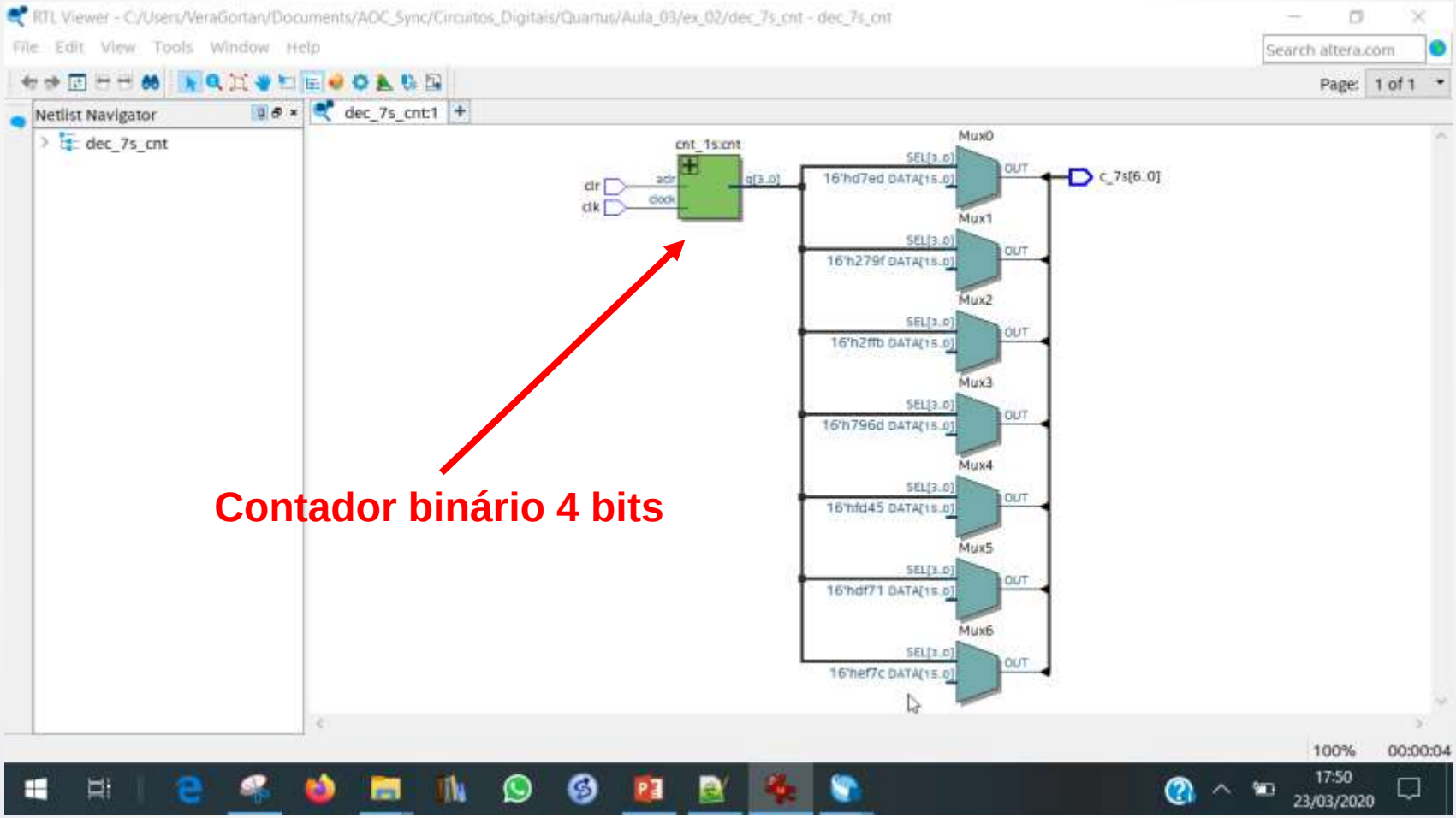
Quartus
Compilar, Simular Testar

- Pronto!
- Você agora pode:
  - Compilar o seu projeto
  - Simular no Quartus
  - Testar com o kit EPM 240
- O próximo slide mostra uma possível simulação funcional no quartus

Quartus
Saída de simulação funcional no Quartus



Quartus
Visão RTL do circuito do Exercício 2



Contador binário 4 bits

Exercício 3

Neste exercício você usará tudo o que aprendeu até aqui e mais a sua **criatividade** para:

Acrescentar um módulo divisor em vhdl ao circuito, o qual divide o clock do EPM240 de 50 MHz (pino 12) para obter um clock de 1 s.

Esse clock de 1s. será conectado à entrada clk do contador do exercício anterior para fazer o display avançar uma posição a cada segundo automaticamente.

Bom divertimento!

Exercício 3
Sugestão para implementar o divisor de frequência

Utilize novamente um LPM_COUNTER, mas desta vez com módulo 50.000.000 e com uma saída de Co (Carry-out) (veja slide 36).

O contador contará até 50.000.000, emitirá um pulso de Co (Carry-out) e retornará a zero.

Com isso você obtém na saída um pulso a cada 50.000.000 de pulsos da entrada, ou seja um pulso a cada segundo para um entrada de 50 MHz.

Assim, a frequência de 50 MHz foi dividida por 50.000.000 produzindo uma saída de 1 Hz!